

图论算法

最小生成树

有两种算法，一般使用kruskal

```

//prim算法求最小生成树的大小
#include<bits/stdc++.h>
using namespace std;
#define re register
#define il inline
il int read()
{
    re int x=0,f=1;char c=getchar();
    while(c<'0' || c>'9'){if(c=='-') f=-1;c=getchar();}
    while(c>='0'&& c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
    return x*f;
}
//快读，不理解的用cin代替即可
#define inf 123456789
#define maxn 5005
#define maxm 200005
struct edge
{
    int v,w,next;
}e[maxm<<1];
//注意是无向图，开两倍数组
int head[maxn],dis[maxn],cnt,n,m,tot,now=1,ans;
//已经加入最小生成树的的点到没有加入的点的最短距离，比如说1和2号节点已经加入了最小生成树，那么dis[3]崩
bool vis[maxn];
//链式前向星加边
il void add(int u,int v,int w)
{
    e[++cnt].v=v;
    e[cnt].w=w;
    e[cnt].next=head[u];
    head[u]=cnt;
}
//读入数据
il void init()
{
    n=read(),m=read();
    for(re int i=1,u,v,w;i<=m;++i)
    {
        u=read(),v=read(),w=read();
        add(u,v,w),add(v,u,w);
    }
}
il int prim()
{

```

```

    //先把dis数组附为极大值
    for(re int i=2;i<=n;++i)
    {
        dis[i]=inf;
    }
    //这里要注意重边，所以要用到min
    for(re int i=head[1];i;i=e[i].next)
    {
        dis[e[i].v]=min(dis[e[i].v],e[i].w);
    }
    while(++tot<n)//最小生成树边数等于点数-1
    {
        re int minn=inf;//把minn置为极大值
        vis[now]=1;//标记点已经走过
        //枚举每一个没有使用的点
        //找出最小值作为新边
        //注意这里不是枚举now点的所有连边，而是1~n
        for(re int i=1;i<=n;++i)
        {
            if(!vis[i]&&minn>dis[i])
            {
                minn=dis[i];
                now=i;
            }
        }
        ans+=minn;
        //枚举now的所有连边，更新dis数组
        for(re int i=head[now];i;i=e[i].next)
        {
            re int v=e[i].v;
            if(dis[v]>e[i].w&&!vis[v])
            {
                dis[v]=e[i].w;
            }
        }
    }
    return ans;
}

int main()
{
    init();
    printf("%d",prim());
    return 0;
}

```



```

//kruskal
#include<bits/stdc++.h>
using namespace std;
#define re register
#define il inline
il int read()
{
    re int x=0,f=1;char c=getchar();
    while(c<'0' || c>'9'){if(c=='-') f=-1;c=getchar();}
    while(c>='0'&& c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
    return x*f;
}
struct Edge
{
    int u,v,w;
}edge[200005];
int fa[5005],n,m,ans,eu,ev,cnt;
il bool cmp(Edge a,Edge b)
{
    return a.w<b.w;
}
//快排的依据（按边权排序）
il int find(int x)
{
    while(x!=fa[x]) x=fa[x]=fa[fa[x]];
    return x;
}
//并查集循环实现模板，及路径压缩，不懂并查集的同学可以戳一戳代码上方的"并查集详解"
il void kruskal()
{
    sort(edge,edge+m,cmp);
    //将边的权值排序
    for(re int i=0;i<m;i++)
    {
        eu=find(edge[i].u), ev=find(edge[i].v);
        if(eu==ev)
        {
            continue;
        }
        //若出现两个点已经联通了，则说明这一条边不需要了
        ans+=edge[i].w;
        //将此边权计入答案
        fa[ev]=eu;
    }
}

```

```

        //将eu、ev合并
        if(++cnt==n-1)
        {
            break;
        }
        //循环结束条件，及边数为点数减一时
    }
}
int main()
{
    n=read(),m=read();
    for(re int i=1;i<=n;i++)
    {
        fa[i]=i;
    }
    //初始化并查集
    for(re int i=0;i<m;i++)
    {
        edge[i].u=read(),edge[i].v=read(),edge[i].w=read();
    }
    kruskal();
    if(cnt!=n-1) cout<<"orz"<<endl;
    else printf("%d",ans);
    return 0;
}

```

最近公共祖先 (LCA)

//利用树链剖分的方法

```
#include<bits/stdc++.h>
using namespace std;
const int N=5e5+5;
struct node{
    int next,to;
}e[N<<1];
int tot,head[N];
int dep[N],siz[N],hson[N],fa[N];
int top[N];
int n,m,s;
void add(int x,int y){
    e[++tot].to=y;
    e[tot].next=head[x];
    head[x]=tot;
}
void dfs1(int x,int f){
    siz[x]=1;
    fa[x]=f;
    dep[x]=dep[f]+1;
    for(int i=head[x];i;i=e[i].next){
        int y=e[i].to;
        if(y!=f){
            dfs1(y,x);
            siz[x]+=siz[y];
            if(siz[hson[x]]<siz[y] || !hson[x]){
                hson[x]=y;
            }
        }
    }
}
void dfs2(int x,int t){
    top[x]=t;
    if(!hson[x])return ;
    dfs2(hson[x],t);
    for(int i=head[x];i;i=e[i].next){
        int y=e[i].to;
        if(y!=fa[x] && y!=hson[x]){
            dfs2(y,y);
        }
    }
}
```

```

}
int LCA(int x,int y){
    while(top[x]!=top[y]){
        if(dep[top[x]]<dep[top[y]]){
            swap(x,y);
        }
        x=fa[top[x]];
    }
    return dep[x]<dep[y]?x:y;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    cin>>n>>m>>s;
    for(int i=1;i<n;i++){
        int x,y;
        cin>>x>>y;
        add(x,y);
        add(y,x);
    }
    dfs1(s,0);
    dfs2(s,s);
    for(int i=1;i<=m;i++){
        int a,b;
        cin>>a>>b;
        cout<<LCA(a,b)<<'\\n';
    }
    return 0;
}

```



```

//使用链式前向星与tarjan算法的结合
#include<bits/stdc++.h>
using namespace std;
const int maxn=5e5+5;
int cnt=1,head[maxn<<1],n,m,s;
struct graph{
    int next,to;
}g[maxn<<1];
void add(int u,int v){
    g[cnt].next=head[u];
    g[cnt].to=v;
    head[u]=cnt++;
}
//问题列表也可以用链式前向星存储
int qcnt=1,qhead[maxn<<1],qindex[maxn<<1];
struct quention{
    int next,to;
}q[maxn<<1];
void addq(int u,int v,int i){
    q[qcnt].next=qhead[u];
    q[qcnt].to=v;
    qindex[qcnt]=i;
    qhead[u]=qcnt++;
}
bool vis[maxn];
int father[maxn];
int ans[maxn];//答案与问题编号
void build(){
    qcnt=cnt=1;
    for(int i=1;i<=n;i++){
        head[i]=qhead[i]=0;
        vis[i]=false;
        father[i]=i;
    }
}
int find(int x){
    father[x]=father[x]==x?x:find(father[x]);
    return father[x];
}
void tarjan(int u,int f){
    vis[u]=true;
    for(int ei=head[u];ei;ei=g[ei].next){

```

```

        int v=g[ei].to;
        if(v!=f){
            tarjan(v,u);
            father[v]=u;
        }
    }
    for(int ei=qhead[u];ei;ei=q[ei].next){
        int v=q[ei].to;
        if(vis[v]) ans[qindex[ei]]=find(v);
    }
}

int main(){
    cin>>n>>m>>s;
    build();
    for(int i=1;i<n;i++){
        int u,v;
        cin>>u>>v;
        add(u,v);
        add(v,u);
    }
    for(int i=1;i<=m;i++){
        int u,v;
        cin>>u>>v;
        addq(u,v,i);
        addq(v,u,i);
    }
    tarjan(s,0);
    for(int i=1;i<=m;i++){
        cout<<ans[i]<<endl;
    }
}

```

树链剖分

```
#include<algorithm>
#include<iostream>
#include<cstdlib>
#include<cstring>
#include<cstdio>

#define Rint register int
#define mem(a,b) memset(a,(b),sizeof(a))
#define Temp template<typename T>
using namespace std;
typedef long long LL;

Temp inline void read(T &x){
    x=0;T w=1,ch=getchar();
    while(!isdigit(ch)&&ch!='-')ch=getchar();
    if(ch=='-')w=-1,ch=getchar();
    while(isdigit(ch))x=(x<<3)+(x<<1)+(ch^'0'),ch=getchar();
    x=x*w;
}

#define mid ((l+r)>>1)
#define lson rt<<1,l,mid
#define rson rt<<1|1,mid+1,r
#define len (r-l+1)

const int maxn=200000+10;
int n,m,r,mod;
// 见题意
int e,beg[maxn],nex[maxn],to[maxn],w[maxn],wt[maxn];
// 链式前向星数组, w[]、wt[]初始点权数组
int a[maxn<<2],laz[maxn<<2];
// 线段树数组、lazy操作
int son[maxn],id[maxn],fa[maxn],cnt,dep[maxn],siz[maxn],top[maxn];
// son[]重儿子编号,id[]新编号,fa[]父亲节点,cnt dfs_clock/dfs序,dep[]深度,siz[]子树大小,top[]当前链顶
int res=0;
// 查询答案

inline void add(int x,int y){//链式前向星加边
    to[++e]=y;
    nex[e]=beg[x];
    beg[x]=e;
}

//----- 以下为线段树
```

```

inline void pushdown(int rt,int len){
    laz[rt<<1]+=laz[rt];
    laz[rt<<1|1]+=laz[rt];
    a[rt<<1]+=laz[rt]*(len-(len>>1));
    a[rt<<1|1]+=laz[rt]*(len>>1);
    a[rt<<1]%=mod;
    a[rt<<1|1]%=mod;
    laz[rt]=0;
}

inline void build(int rt,int l,int r){
    if(l==r){
        a[rt]=wt[l];
        if(a[rt]>mod)a[rt]%=mod;
        return;
    }
    build(lson);
    build(rson);
    a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
}

inline void query(int rt,int l,int r,int L,int R){
    if(L<=l&&r<=R){res+=a[rt];res%=mod;return;}
    else{
        if(laz[rt])pushdown(rt,len);
        if(L<=mid)query(lson,L,R);
        if(R>mid)query(rson,L,R);
    }
}

inline void update(int rt,int l,int r,int L,int R,int k){
    if(L<=l&&r<=R){
        laz[rt]+=k;
        a[rt]+=k*len;
    }
    else{
        if(laz[rt])pushdown(rt,len);
        if(L<=mid)update(lson,L,R,k);
        if(R>mid)update(rson,L,R,k);
        a[rt]=(a[rt<<1]+a[rt<<1|1])%mod;
    }
}

```

//-----以上为线段树

```

inline int qRange(int x,int y){
    int ans=0;
    while(top[x]!=top[y]){//当两个点不在同一条链上
        if(dep[top[x]]<dep[top[y]])swap(x,y);//把x点改为所在链顶端的深度更深的那个点
        res=0;
        query(1,1,n,id[top[x]],id[x]);//ans加上x点到x所在链顶端 这一段区间的点权和
        ans+=res;
        ans%=mod;//按题意取模
        x=fa[top[x]);//把x跳到x所在链顶端的那个点的上面一个点
    }
    //直到两个点处于一条链上
    if(dep[x]>dep[y])swap(x,y);//把x点深度更深的那个点
    res=0;
    query(1,1,n,id[x],id[y]);//这时再加上此时两个点的区间和即可
    ans+=res;
    return ans%mod;
}

```

```

inline void updRange(int x,int y,int k){//同上
    k%=mod;
    while(top[x]!=top[y]){
        if(dep[top[x]]<dep[top[y]])swap(x,y);
        update(1,1,n,id[top[x]],id[x],k);
        x=fa[top[x]];
    }
    if(dep[x]>dep[y])swap(x,y);
    update(1,1,n,id[x],id[y],k);
}

```

```

inline int qSon(int x){
    res=0;
    query(1,1,n,id[x],id[x]+siz[x]-1);//子树区间右端点为id[x]+siz[x]-1
    return res;
}

```

```

inline void updSon(int x,int k){//同上
    update(1,1,n,id[x],id[x]+siz[x]-1,k);
}

```

```

inline void dfs1(int x,int f,int deep){//x当前节点，f父亲，deep深度
    dep[x]=deep;//标记每个点的深度
    fa[x]=f;//标记每个点的父亲
    siz[x]=1;//标记每个非叶子节点的子树大小
}

```

```

int maxson=-1;//记录重儿子的儿子数
for(Rint i=beg[x];i;i=nex[i]){
    int y=to[i];
    if(y==f)continue;//若为父亲则continue
    dfs1(y,x,deep+1);//dfs其儿子
    siz[x]+=siz[y];//把它的儿子数加到它身上
    if(siz[y]>maxson)son[x]=y,maxson=siz[y];//标记每个非叶子节点的重儿子编号
}
}

inline void dfs2(int x,int topf){//x当前节点，topf当前链的最顶端的节点
    id[x]++;cnt;//标记每个点的新编号
    wt[cnt]=w[x];//把每个点的初始值赋到新编号上来
    top[x]=topf;//这个点所在链的顶端
    if(!son[x])return;//如果没有儿子则返回
    dfs2(son[x],topf);//按先处理重儿子，再处理轻儿子的顺序递归处理
    for(Rint i=beg[x];i;i=nex[i]){
        int y=to[i];
        if(y==fa[x]||y==son[x])continue;
        dfs2(y,y);//对于每一个轻儿子都有一条从它自己开始的链
    }
}

int main(){
    read(n);read(m);read(r);read(mod);
    for(Rint i=1;i<=n;i++)read(w[i]);
    for(Rint i=1;i<n;i++){
        int a,b;
        read(a);read(b);
        add(a,b);add(b,a);
    }
    dfs1(r,0,1);
    dfs2(r,r);
    build(1,1,n);
    while(m--){
        int k,x,y,z;
        read(k);
        if(k==1){
            read(x);read(y);read(z);
            updRange(x,y,z);
        }
        else if(k==2){
            read(x);read(y);

```

```
        printf("%d\n",qRange(x,y));
    }
    else if(k==3){
        read(x);read(y);
        updSon(x,y);
    }
    else{
        read(x);
        printf("%d\n",qSon(x));
    }
}
}
```

最大路匹配

```
#include<bits/stdc++.h>
using namespace std;
const int N=510;
vector<int>g[N];
bool vis[N];
int match[N];
bool dfs(int u){
    for(int v:g[u]){
        if(!vis[v]){
            vis[v]=1;
            if(match[v]==0||dfs(match[v])){
                match[v]=u;
                return true;
            }
        }
    }
    return false;
}
int main(){
    int n,m,ans=0;
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        int x;
        cin>>x;
        for(int j=1;j<=x;j++){
            int y;
            cin>>y;
            g[i].push_back(y);
        }
    }
    for(int i=1;i<=n;i++){
        memset(vis,0,sizeof(vis));
        if(dfs(i)) ans++;
    }
    cout<<ans;
}
```


Dijkstra算法

//小根堆实现较为简单，这里采用以点为复杂度的反向索引堆的优化，一般不会用

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=100010;
const int inf=1e8;
struct graph{
    int next,to,w;
}g[maxn*2];
int head[maxn],cnt,n,m;
int heap[maxn];//反向索引堆
int where[maxn];//表示点在堆上位置，初始化为-1
int heapSize;
int dis[maxn];
void build(int n){
    cnt=1;
    heapSize=0;
    for(int i=1;i<=n;i++){
        heap[i]=0;
        where[i]=-1;
        head[i]=0;
        dis[i]=inf;
    }
}
void add(int u,int v,int w){
    g[cnt].next=head[u];
    g[cnt].to=v;
    g[cnt].w=w;
    head[u]=cnt++;
}
void Swap(int i,int j){
    swap(heap[i],heap[j]);
    where[heap[i]]=i;
    where[heap[j]]=j;
}
void heapInsert(int i){
    while(dis[heap[i]]<dis[heap[(i-1)/2]]){
        Swap(i,(i-1)/2);
        i=(i-1)/2;
    }
}
void addorupdateorignore(int v,int c){
```

```

    if(where[v]==-1){
        heap[heapSize]=v;
        where[v]=heapSize++;
        dis[v]=c;
        heapInsert(where[v]);
    }else if(where[v]>=0){
        dis[v]=min(dis[v],c);
        heapInsert(where[v]);
    }
}

void heapify(int i){
    int l=1;
    while(l<heapSize){
        int best=l+1<heapSize&&dis[heap[l+1]]<dis[heap[l]]?l+1:l;
        best=dis[heap[best]]<dis[heap[i]]?best:i;
        if(best==i) break;
        Swap(best,i);
        i=best;
        l=i*2+1;
    }
}

bool isEmpty(){
    return heapSize==0;
}

int pop(){
    int ans=heap[0];
    swap(heap[0],heap[--heapSize]);
    heapify(0);
    where[ans]=-2;
    return ans;
}

int main(){
    cin>>n>>m;
    int s;
    cin>>s;
    build(n);
    for(int i=1;i<=m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        add(u,v,w);
    }
    addOrUpdateIgnore(s,0);
    while(!isEmpty()){

```

```

    int u=pop();
    for(int ei=head[u];ei>0;ei=g[ei].next){
        addorupdateorignore(g[ei].to,dis[u]+g[ei].w);
    }
}
int ans=-inf;
for(int i=1;i<=n;i++){
    /*if(dis[i]==inf){
        cout<<-1<<endl;
    }
    ans=max(ans,dis[i]);*/
    cout<<dis[i]<<' ';
}
//cout<<ans<<endl;
}

```

分层图最短路

//无论是在bfs上分层还是dijkstra上，本质其实就是把原图上的点与它的不同状态整合出更多的点
//难点在于如何将点上的状态给储存下来，以及如何转移

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=50010;
const int inf=1e8;
struct dot{
    int x,cost,k;
};
struct graph{
    int next,to,w;
}g[maxn<<1];
struct compare{
    bool operator()(dot x,dot y){
        return x.cost>y.cost;
        // 小根堆
    }
};
int head[maxn<<1],cnt=1,n,m,k,dis[maxn][20];
bool vis[maxn][20];
priority_queue<dot,vector<dot>,compare>q;
void add(int u,int v,int w){
    g[cnt].next=head[u];
    g[cnt].to=v;
    g[cnt].w=w;
    head[u]=cnt++;
}
int main(){
    cin>>n>>m>>k;
    int s,t;
    cin>>s>>t;
    for(int i=1;i<=m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        add(u,v,w);
        add(v,u,w);
    }
    for(int i=0;i<=n;i++){
        for(int j=0;j<=k;j++){
            dis[i][j]=inf;
        }
    }
```

```

}
dis[s][0]=0;
q.push({s,0,0});
while(!q.empty()){
    int xn=q.top().x;
    int cost=q.top().cost;
    int kn=q.top().k;
    q.pop();
    if(vis[xn][kn]) continue;
    vis[xn][kn]=1;
    if(xn==t){
        cout<<cost<<endl;
        return 0;
    }
    for(int i=head[xn];i>0;i=g[i].next){
        int v=g[i].to;
        if(kn<k&&cost<dis[v][kn+1]){
            dis[v][kn+1]=cost;
            q.push({v,cost,kn+1});
        }
        if(cost+g[i].w<dis[v][kn]){
            dis[v][kn]=cost+g[i].w;
            q.push({v,cost+g[i].w,kn});
        }
    }
}
}
}
/*
struct Compare {
    bool operator()(const State& a, const State& b) {
        return a.cost > b.cost; // 小根堆
    }
};

priority_queue<State, vector<State>, Compare> q;
这样子定义也可以
*/

```

双向广搜

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll n,m;
ll an[45];
ll l[1<<20+1],r[1<<20+1];
ll f(int i,int e,ll s,ll w,ll a[],ll j){
    if(s>w) return j;
    if(i==e) a[j++]=s;
    else {
        j=f(i+1,e,s,w,a,j);
        j=f(i+1,e,s+an[i],w,a,j);
    }
    return j;
}
ll compute(){
    ll lsize=f(0,n>>1,0,m,l,0);
    ll rsize=f(n>>1,n,0,m,r,0);
    sort(l,l+lsize);
    sort(r,r+rsize);
    ll ans=0;
    for(int i=lsize-1,j=0;i>=0;i--){
        while(j<rsize&&1[i]+r[j]<=m){
            j++;
        }
        ans+=j;
    }
    return ans;
}
int main(){
    cin>>n>>m;
    for(int i=0;i<n;i++) cin>>an[i];
    sort(an,an+n);
    cout<<compute();
}
```

拓扑排序

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=100001;
int cnt,n,m,head[maxn],indegree[maxn],per[maxn];

struct graph{
    int next,to;
}g[maxn];

void add(int u,int v){
    g[cnt].next=head[u];
    g[cnt].to=v;
    head[u]=cnt++;
}

bool topsort(){
    //queue<int> q; // 如果把队列改成小根堆的话，就可以使队列结果为字典序最小排列
    priority_queue<int,vector<int>,greater<int>>q;
    for(int i=1;i<=n;i++){
        if(indegree[i]==0){
            q.push(i);
        }
    }
    int fill=0;
    while(!q.empty()){
        int cur=q.top();
        q.pop();
        per[++fill]=cur;
        for(int ei=head[cur];ei!=0;ei=g[ei].next){
            if(--indegree[g[ei].to]==0){
                q.push(g[ei].to);
            }
        }
    }
    return fill==n;
}

int main(){
    cin>>n>>m;
    cnt=1;
    memset(head,0,sizeof(head)); // 初始化head数组
    memset(indegree,0,sizeof(indegree)); // 初始化入度数组
```

```
for(int i=1;i<=m;i++){
    int u,v;
    cin>>u>>v;
    add(u,v);
    indegree[v]++;
}

if(!tpsort()){
    cout<<-1<<endl;
}else {
    for(int i=1;i<=n;i++){
        cout<<per[i]<<' ';
    }
    cout<<endl;
}
return 0;
}
```


tarjan(强连通分量)

//强连通分量求解

```
int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp;
int scc[N], sc; // 结点 i 所在 SCC 的编号
int sz[N];      // 强连通 i 的大小

void tarjan(int u) {
    low[u] = dfn[u] = ++dfncnt, s[++tp] = u, in_stack[u] = 1;
    for (int i = h[u]; i; i = e[i].nex) {
        const int &v = e[i].t;
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (in_stack[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] == low[u]) {
        ++sc;
        do {
            scc[s[tp]] = sc;
            sz[sc]++;
            in_stack[s[tp]] = 0;
        } while (s[tp--] != u);
    }
}
```

```

#include <bits/stdc++.h>
#define N 100010 // 定义数组最大长度，适配题目中 $N \leq 1e4$ 的范围（留有余量）
using namespace std;
struct graph{
    int t,nex;
}e[N*2];
//强连通分量求解
int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp,cnt=1,h[N*2];
int scc[N], sc; // 结点 i 所在 SCC 的编号
int sz[N];      // 强连通 i 的大小
int out[N];
void add(int u,int v){
    e[cnt].nex=h[u];
    e[cnt].t=v;
    h[u]=cnt++;
}
//求每个分量出度
for (int i = 1; i <= n; i++) {
    // 遍历i的所有邻接节点v
    for (int j = h[i]; j ; j=e[i].nex) {
        int v = e[i].t
        // 如果i和v属于不同的强连通分量 → 说明缩点后i的分量有一条出边到v的分量
        if (scc[i]!=scc[v]) {
            out[scc[i]]++; // i所属分量的出度+1
        }
    }
}
}

```

边双连通分量

```
#include <algorithm>
#include <iostream>
#include <stack>
#include <vector>

using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
    int to, nt;
} e[M << 2];

int hd[N << 1], tot = 1;

void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

int bfn cnt, sum;
int dfn[N], low[N], bcc[N], bcc_cnt;
bool vis[N];
vector<vector<int>> ans;
stack<int> st;
void tarjan(int u, int in) {
    low[u] = dfn[u] = ++dfn cnt;
    st.push(u), vis[u] = true;
    for (int i = hd[u]; i; i = e[i].nt) {
        int v = e[i].to;
        if (i == (in ^ 1)) continue;
        if (!dfn[v])
            tarjan(v, i), low[u] = min(low[u], low[v]);
        else if (vis[v])
            low[u] = min(low[u], dfn[v]);
    }
    if (dfn[u] == low[u]) {
        bcc_cnt++;
        bcc[u] = bcc_cnt;
        vector<int> t;
        t.push_back(u);
        while (st.top() != u)
```

```

        t.push_back(st.top()), bcc[st.top()]=bcc_cnt,vis[st.top()] = false, st.pop();
    st.pop(), ans.push_back(t);
}
}

```

割边

```

void tarjan(int u,int pre) {
    low[u] = dfn[u] = ++dfncnt;
    for (int ei = head[u]; ei; ei = e[ei].nex) {
        if((ei^1)==pre) continue;
        const int &v = e[ei].t;
        if (!dfn[v]) {
            tarjan(v,ei);

            low[u] = min(low[u], low[v]);
            if(low[v]>dfn[u]){
                cutedge[ei>>1]=1;//无向图割边
                ans1++;
            }
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}
}

```

点双连通分量

```
void tarjan(int u) {
    dfn[u] = low[u] = ++bcc_cnt, sta[++top] = u;
    if (u == root && hd[u] == 0) { //root=i;
        dcc[++cnt].push_back(u);
        return;
    }
    int f = 0;
    for (int i = hd[u]; i; i = e[i].nt) {
        int v = e[i].to;
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u]) {
                if (++f > 1 || u != root) cut[u] = true;
                cnt++;
                do dcc[cnt].push_back(sta[top--]);
                while (sta[top + 1] != v);
                dcc[cnt].push_back(u);
            }
        } else
            low[u] = min(low[u], dfn[v]);
    }
}
```